**AWS CI/CD PIPELINE TO HOST A STATIC WEBSITE**

**Using GitHub, AWS CodePipeline, AWS CodeBuild and Amazon S3**

---

**1. Aim**

To design and implement an automated **CI/CD pipeline on Amazon Web Services (AWS)** for deploying and hosting a static website using **GitHub, AWS CodePipeline, AWS CodeBuild and Amazon S3**.

The pipeline automatically detects changes pushed to the GitHub repository, builds the website using AWS CodeBuild, and deploys the updated website to an Amazon S3 bucket.

---

**2. Objectives**

The objectives of this experiment are:

1. To create a static website using HTML, CSS and JavaScript.
2. To store the website source code in a GitHub repository.
3. To configure an AWS CodeBuild project.
4. To configure AWS CodePipeline for continuous integration and deployment.
5. To connect GitHub with AWS CodePipeline.
6. To automatically build the website whenever changes are pushed.
7. To deploy the website to an Amazon S3 bucket.
8. To enable static website hosting on Amazon S3.
9. To verify automatic deployment after modifying the source code.

---

**3. Technologies Used**

| Technology | Purpose |
|---|---|
| GitHub | Source code repository |
| AWS CodePipeline | CI/CD pipeline orchestration |
| AWS CodeBuild | Build and package the website |
| Amazon S3 | Website hosting and deployment |
| AWS IAM | Access and permissions management |
| Amazon CloudWatch | Build log monitoring |

| Technology | Purpose |
| --- | --- |
| HTML | Website structure |
| CSS | Website styling |
| JavaScript | Website functionality |

---

**4. System Architecture**

The CI/CD pipeline implemented in this experiment follows the workflow below:

Developer

│

│ Push Code

▼

GitHub Repository

│

│ Trigger

▼

AWS CodePipeline

│

│ Build

▼

AWS CodeBuild

│

│ Build Artifact

▼

Amazon S3

│

│ Static Website Hosting

▼

Deployed Website

**Pipeline Flow**

Code Change

↓

GitHub Push

↓

CodePipeline Trigger

↓

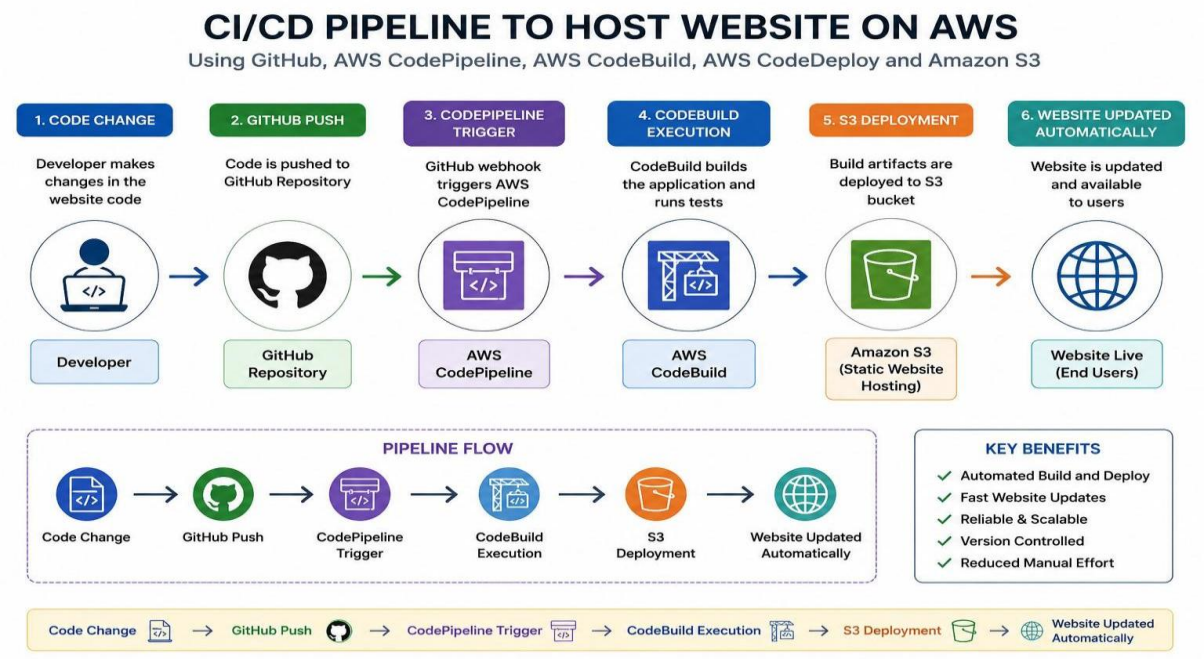
CodeBuild Execution

↓

S3 Deployment

↓

Website Updated Automatically



**5. Project Structure**

The static website used for this experiment contains the following files:

```
my-cicd-project/
│
├── index.html
│
├── css/
│   └── style.css
│
├── js/
│   └── script.js
│
├── images/
│
└── buildspec.yml
```

**Description**

- index.html – Main webpage.
- css/style.css – Styling for the website.
- js/script.js – JavaScript functionality.
- images/ – Directory for website images.
- buildspec.yml – Instructions used by AWS CodeBuild during the build process.

---

**6. Buildspec File**

The buildspec.yml file is placed in the root directory of the GitHub repository.

```
version: 0.2


phases:
 install:
  commands:
   - echo "Starting AWS CodeBuild..."
```

```
 pre_build:
  commands:
   - echo "Preparing website files..."

 build:
  commands:
   - echo "Building static website..."
   - echo "Build completed successfully."

 post_build:
  commands:
   - echo "Preparing deployment artifacts..."

artifacts:
 files:
  - '**/*'
```

The buildspec file defines the commands executed by AWS CodeBuild and specifies the files that are included in the build artifact.

---

**7. Procedure**

**Step 1: Create the Website**

Create a static website using HTML, CSS and JavaScript.

The project should contain:

index.html

css/style.css

js/script.js

images/

buildspec.yml

The website can contain a homepage describing the AWS CI/CD pipeline.

---

**Step 2: Create a GitHub Repository**

Create a new repository on GitHub.

Example repository name:

my-cicd-project

Upload or push the following files:

index.html

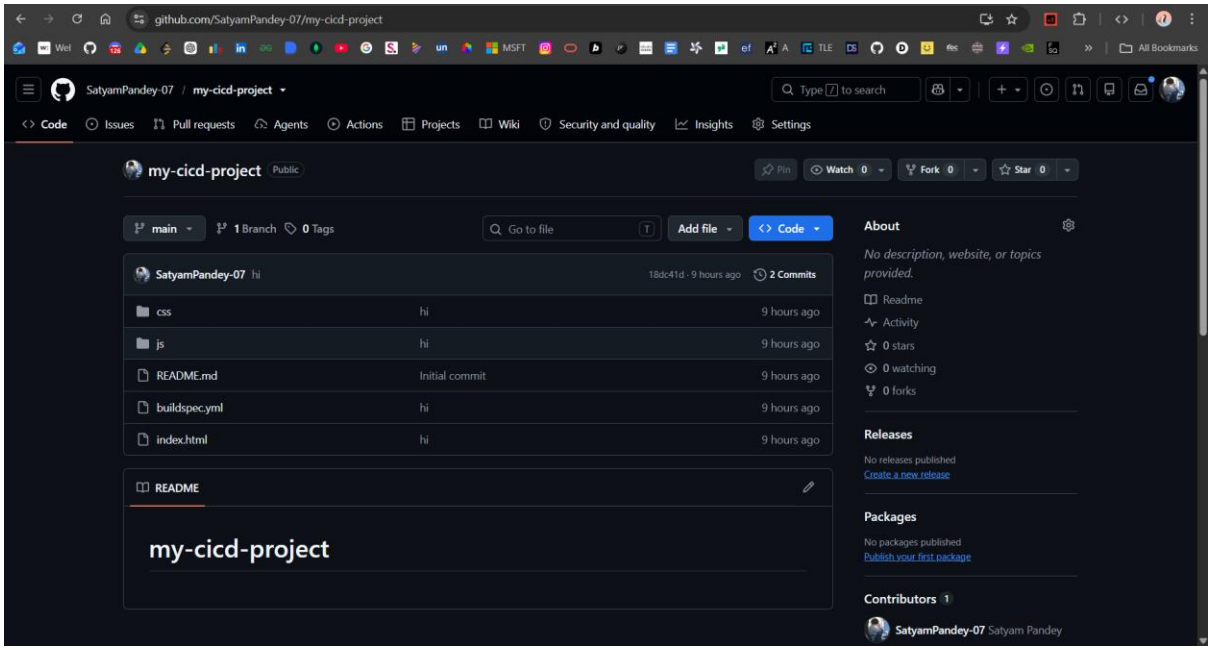
buildspec.yml

css/

js/

images/

The GitHub repository acts as the source code repository for the CI/CD pipeline.




---

**Step 3: Push the Project to GitHub**

Initialize Git in the project directory and push the source code to GitHub.

Example commands:

git init

git add .

git commit -m "Add AWS CI/CD website"

git branch -M main

git remote add origin https://github.com/YOUR_USERNAME/my-cicd-project.git

git push -u origin main

After pushing, verify that all project files are visible in the GitHub repository.

---

**8. Create AWS CodeBuild Project**

**Step 4: Open AWS CodeBuild**

Log in to the AWS Management Console.

Search for:

CodeBuild

Open **AWS CodeBuild** and select:

Build projects → Create build project

---

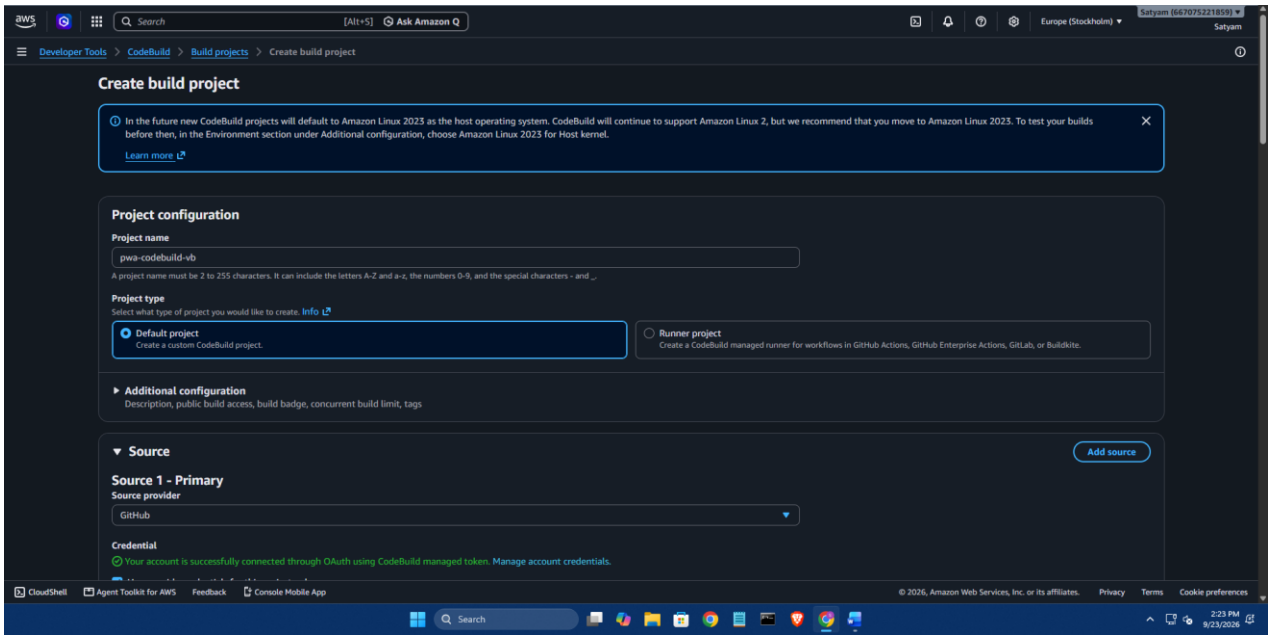
**Step 5: Configure Project Name**

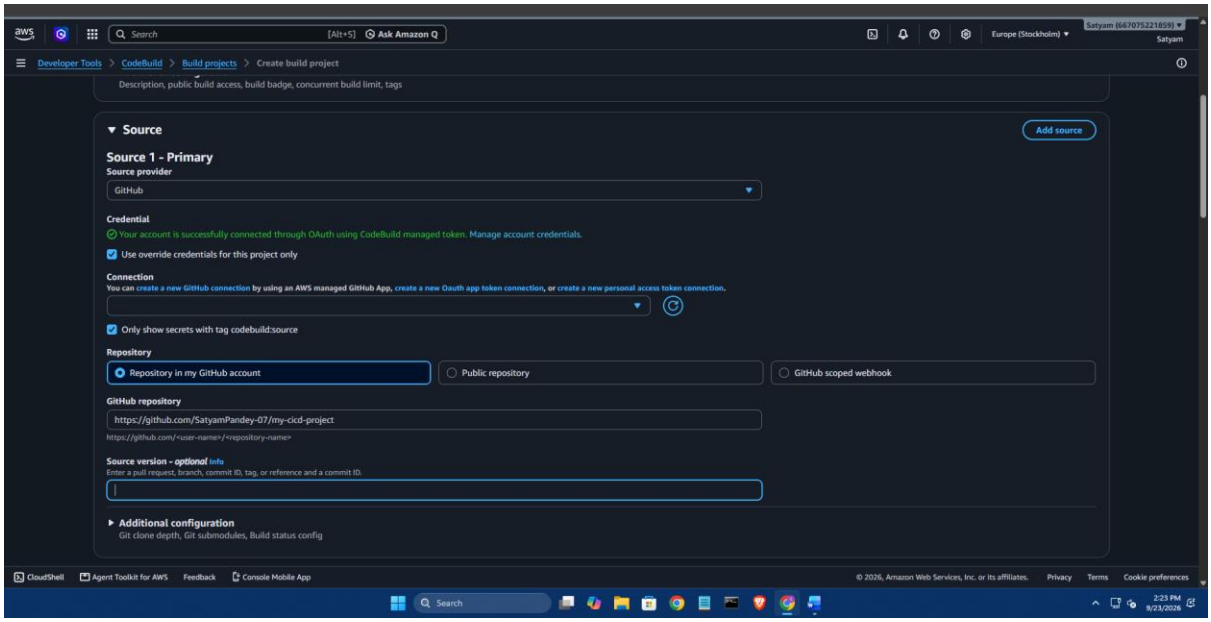
Enter the project name.

Example:

my-cicd-project

The project name uniquely identifies the CodeBuild project.






---

**9. Configure Source**

**Step 6: Select Source Provider**

Under the **Source** section:

Select:

Source provider → GitHub

Connect the GitHub account using the available GitHub authentication/OAuth process.

Authorize AWS CodeBuild when prompted.

The GitHub repository will then become available for selection.

---

**Step 7: Select Repository**

Select:

Repository in my GitHub account

Choose the repository:

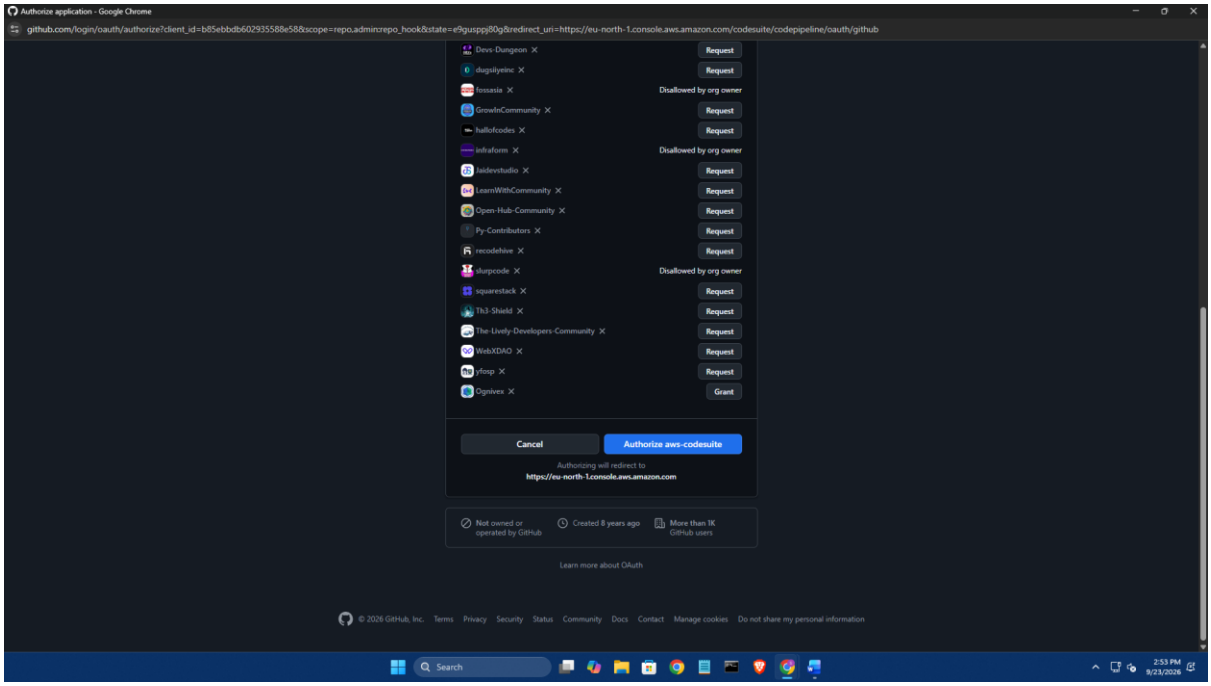
my-cicd-project

Select the branch containing the source code.

For example:

main

The selected GitHub repository becomes the primary source for the CodeBuild project.




---

**10. Configure Environment**

**Step 8: Configure Build Environment**

Scroll to the **Environment** section.

Select:

Environment image:

Managed image

Select:

Running mode:

Instance

Select:

Operating system:

Amazon Linux

For the runtime, use:

Base

Use the AWS-managed CodeBuild image.

---

**Step 9: Configure Service Role**

Under **Service role**, select:

New service role

AWS will automatically create a service role for CodeBuild.

Example:

codebuild-my-cicd-project-service-role

The service role allows CodeBuild to interact with the AWS resources required during the build.

---

**11. Configure Buildspec**

**Step 10: Select Buildspec File**

Scroll to the **Buildspec** section.

Select:

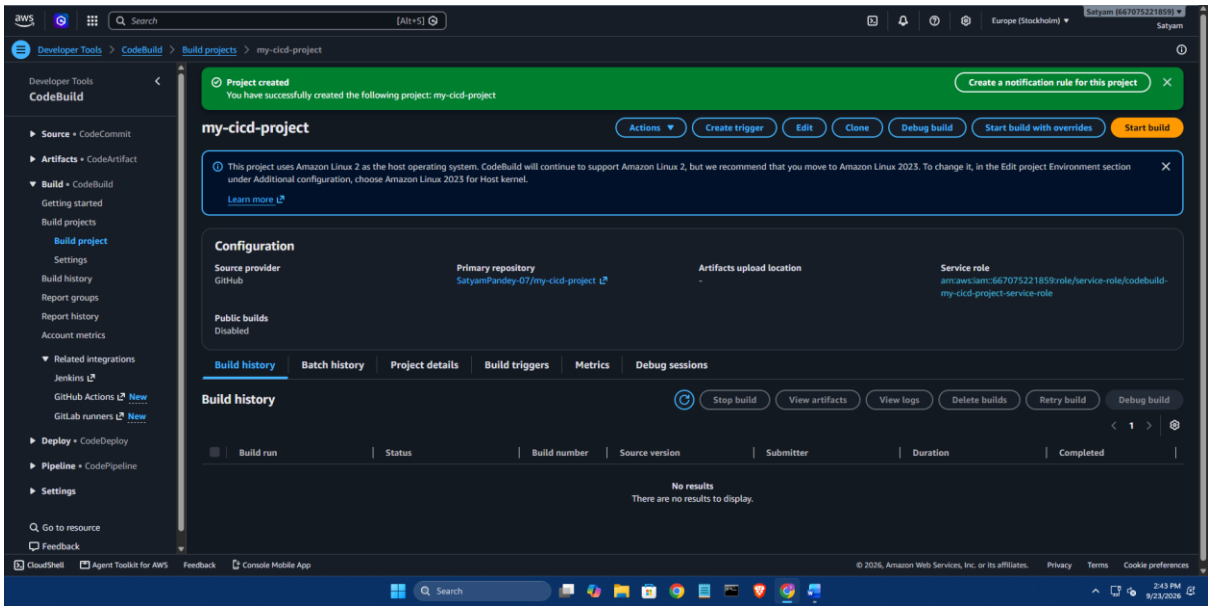
Use a buildspec file

Use:

buildspec.yml

The file is already present in the root directory of the GitHub repository.

AWS CodeBuild will read the instructions from this file during execution.




---

**12. Configure Artifacts**

**Step 11: Configure Artifacts**

For this CodeBuild project, select:

No artifacts

The pipeline will handle the artifacts during the CodePipeline workflow.

The buildspec file still specifies the files that can be used as build output when CodeBuild is invoked by CodePipeline.

---

**13. Configure Logs**

**Step 12: Enable CloudWatch Logs**

Under the **Logs** section, enable:

CloudWatch logs

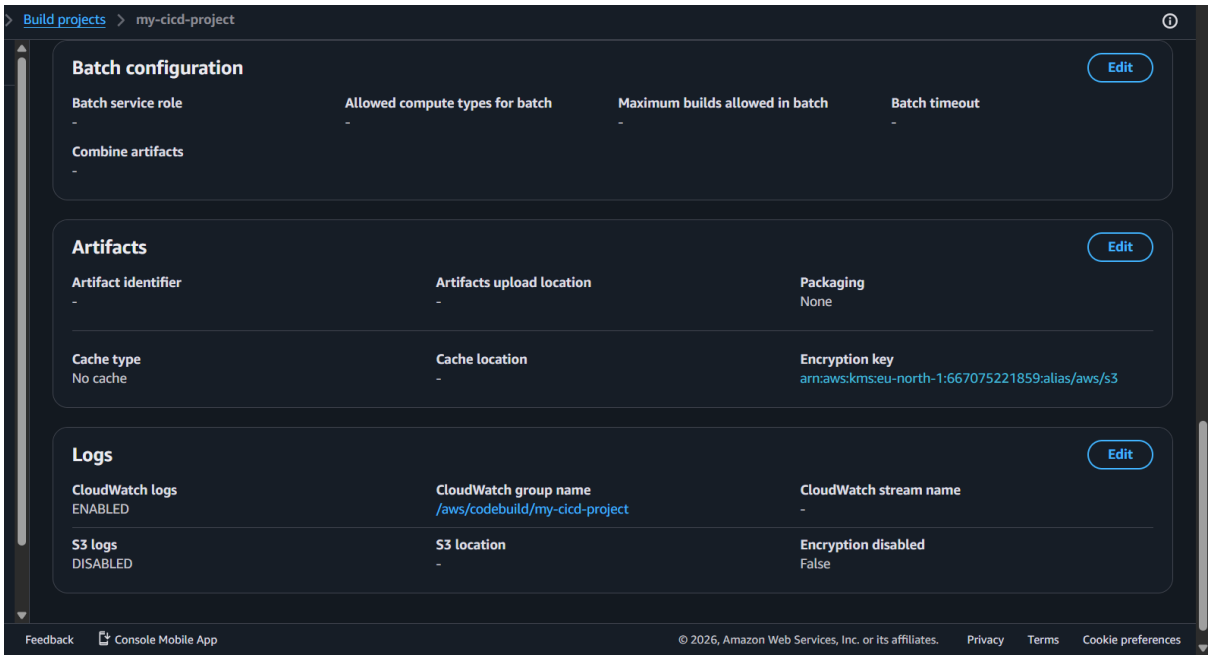
The default log group can be used, for example:

/aws/codebuild/my-cicd-project

S3 build logs are not required for this experiment.

After reviewing the configuration, click:

Create build project




---

**14. Verify CodeBuild Project**

**Step 13: Verify Project Creation**

After creating the project, AWS CodeBuild displays the project details.

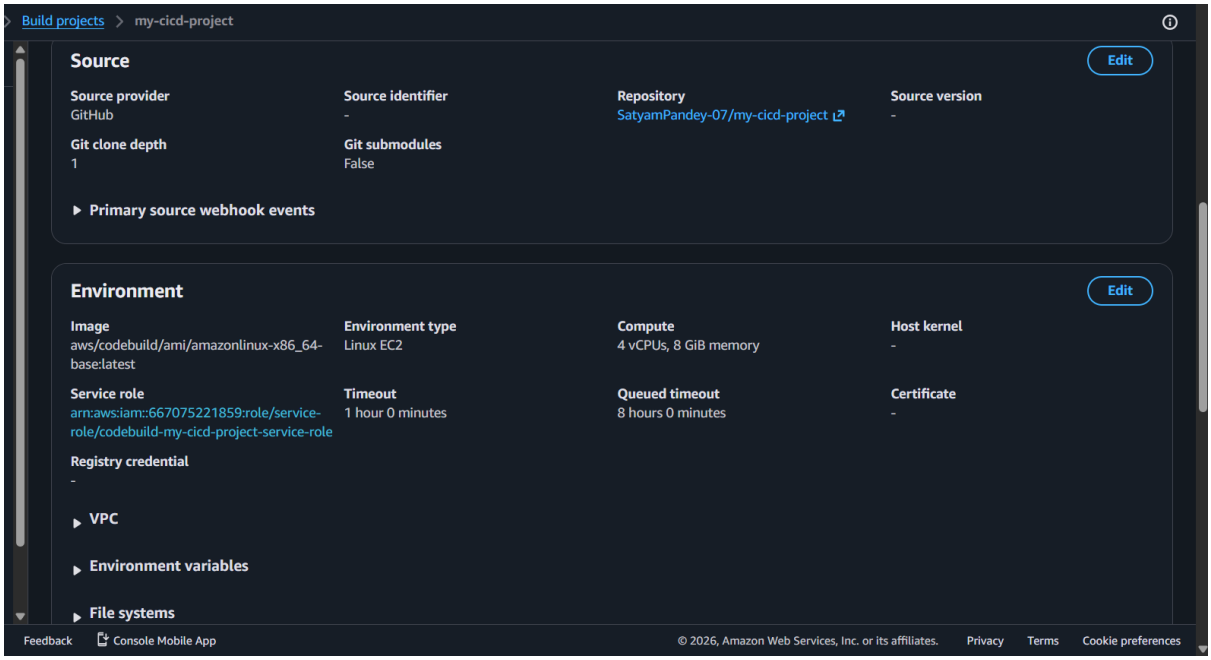
Verify that the project has been successfully created.

The CodeBuild project should show:

Source: GitHub

Environment: Managed Image

Buildspec: buildspec.yml




---

**15. Create AWS CodePipeline**

**Step 14: Open CodePipeline**

Search AWS for:

CodePipeline

Open:

AWS CodePipeline → Pipelines

Select:

Create pipeline

In the newer AWS interface, select:

Build custom pipeline

This allows the individual Source, Build and Deploy stages to be configured manually.

**16. Configure Pipeline Settings**

**Step 15: Configure Pipeline**

Enter a pipeline name.

Example:

my-cicd-pipeline

Select:

New service role

AWS will create the required CodePipeline service role.

The pipeline stores and passes artifacts between its different stages.

---

**17. Configure Artifact Store**

**Step 16: Configure Artifact Store**

Under advanced settings, configure the artifact store.

Use an Amazon S3 bucket for storing CodePipeline artifacts.

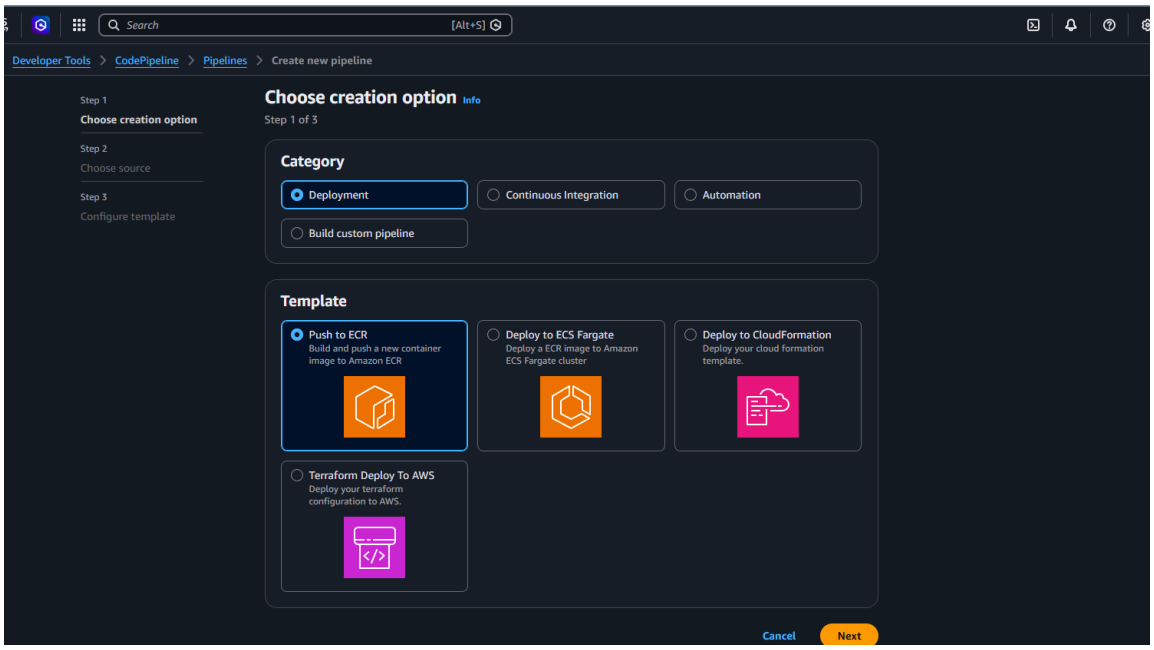
Example:

store-build-artifact-vb

Use the default AWS-managed encryption key unless a different key is required by the assignment.

Click:

Next




---

**18. Configure Source Stage**

**Step 17: Add GitHub Source**

For the Source stage:

Select:

Source provider → GitHub

Connect/authenticate your GitHub account.

Select:

Repository:

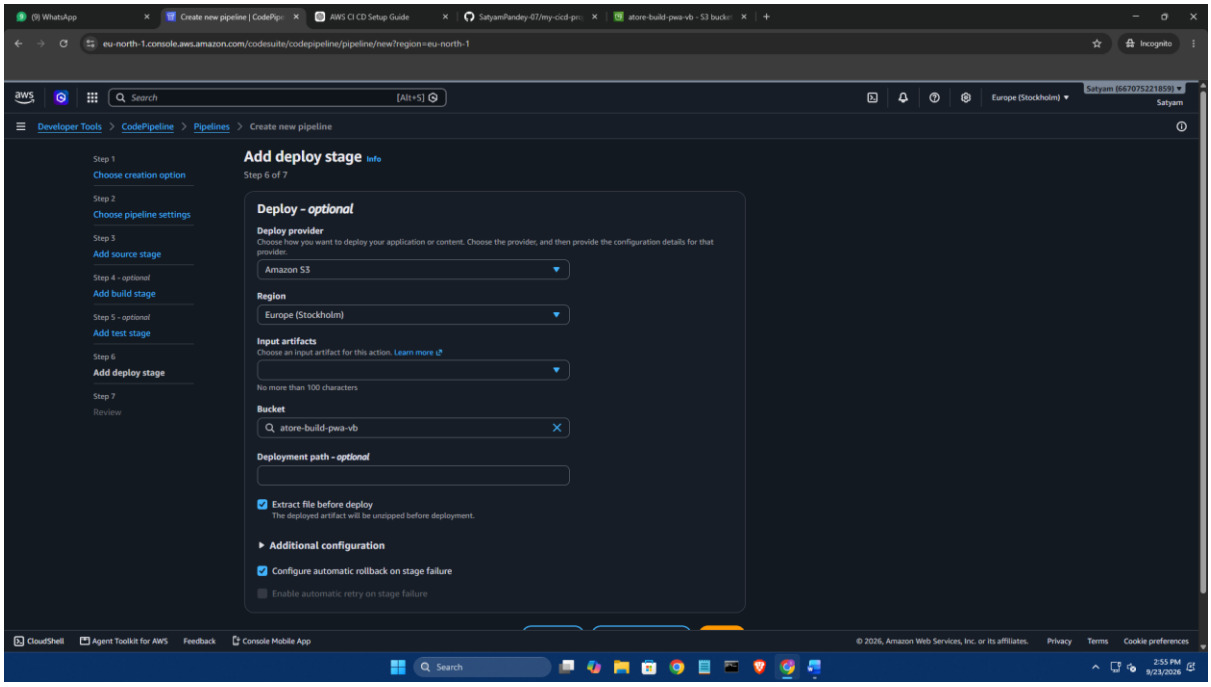
my-cicd-project

Select the branch:

main

Enable automatic change detection if available.

This allows CodePipeline to automatically detect new changes pushed to the repository.




---

**19. Configure Build Stage**

**Step 18: Add Build Stage**

Select:

Build provider

→ Other build providers

→ AWS CodeBuild

Select the CodeBuild project created earlier:

my-cicd-project

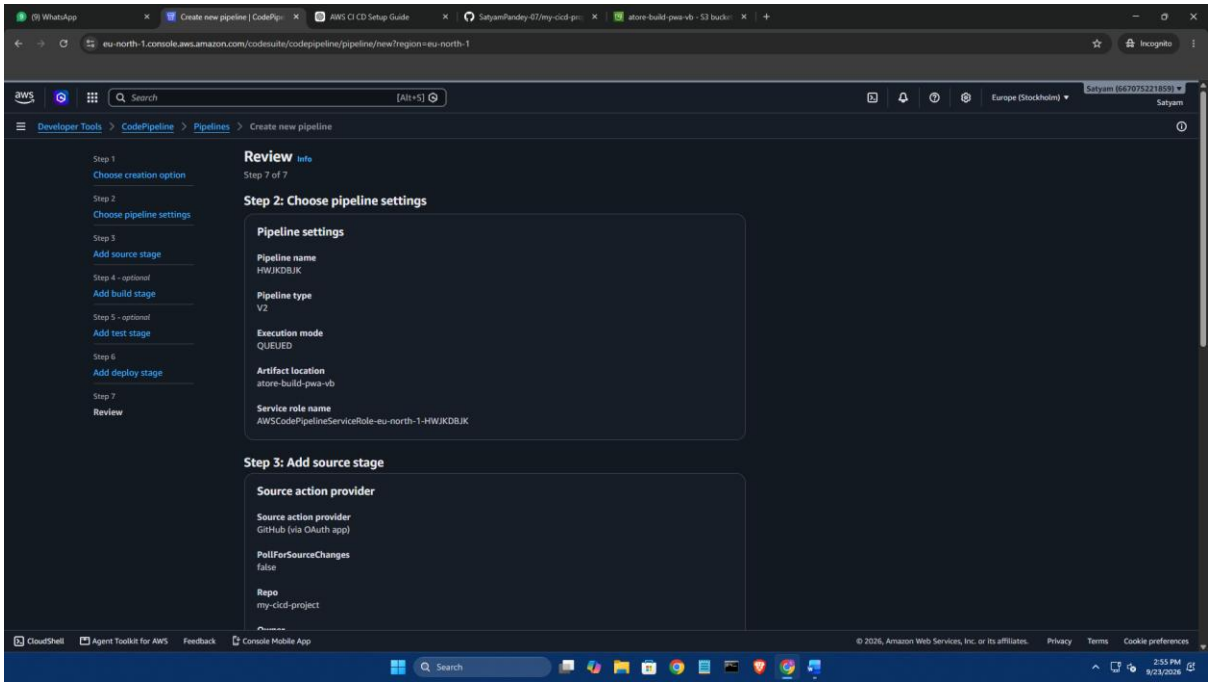
The SourceArtifact generated from the GitHub source stage is provided as the input to CodeBuild.

Use:

Build type: Single build

Click:

Next




---

**20. Configure Deploy Stage**

**Step 19: Add Deployment Stage**

For the deployment stage, select:

Deploy provider → Amazon S3

Select the S3 deployment bucket.

Example:

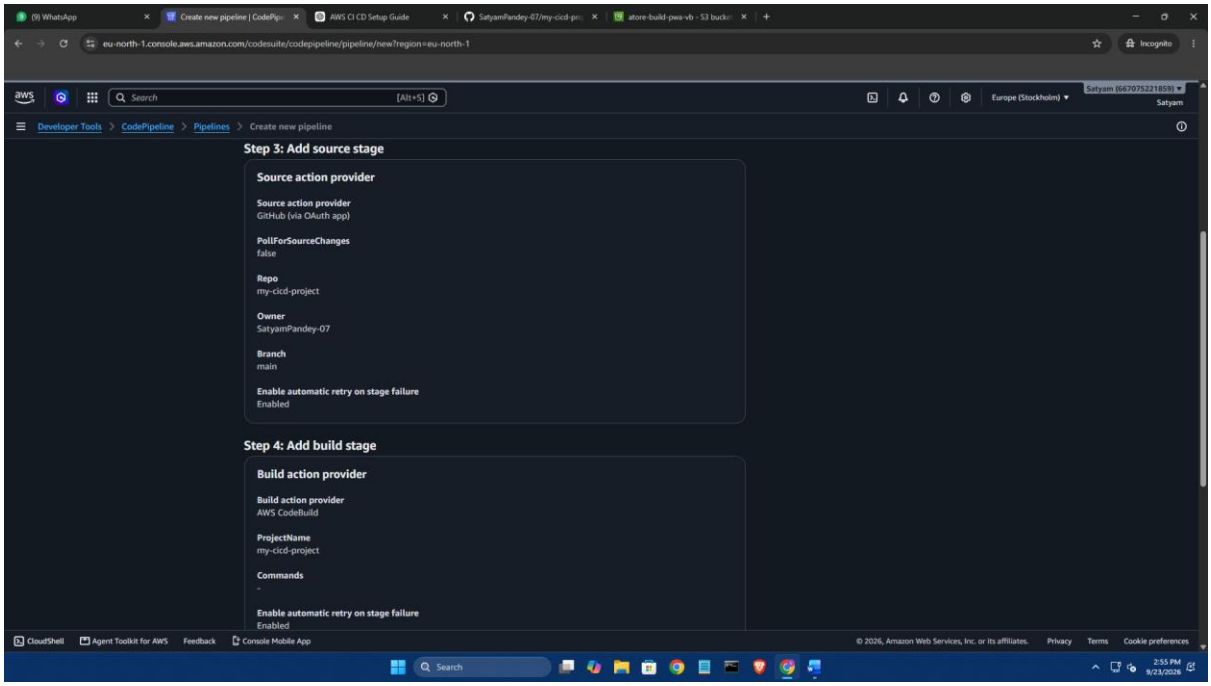
deployment-bucket-pwa-vb

Select the input artifact produced by the build stage.

Enable the option to extract the artifact before deployment if available.

Click:

Next




---

**21. Create Pipeline**

**Step 20: Review and Create Pipeline**

Review the complete pipeline configuration.

The pipeline should contain three major stages:

SOURCE

↓

BUILD

↓

DEPLOY

Where:

GitHub

↓

AWS CodePipeline

↓

AWS CodeBuild

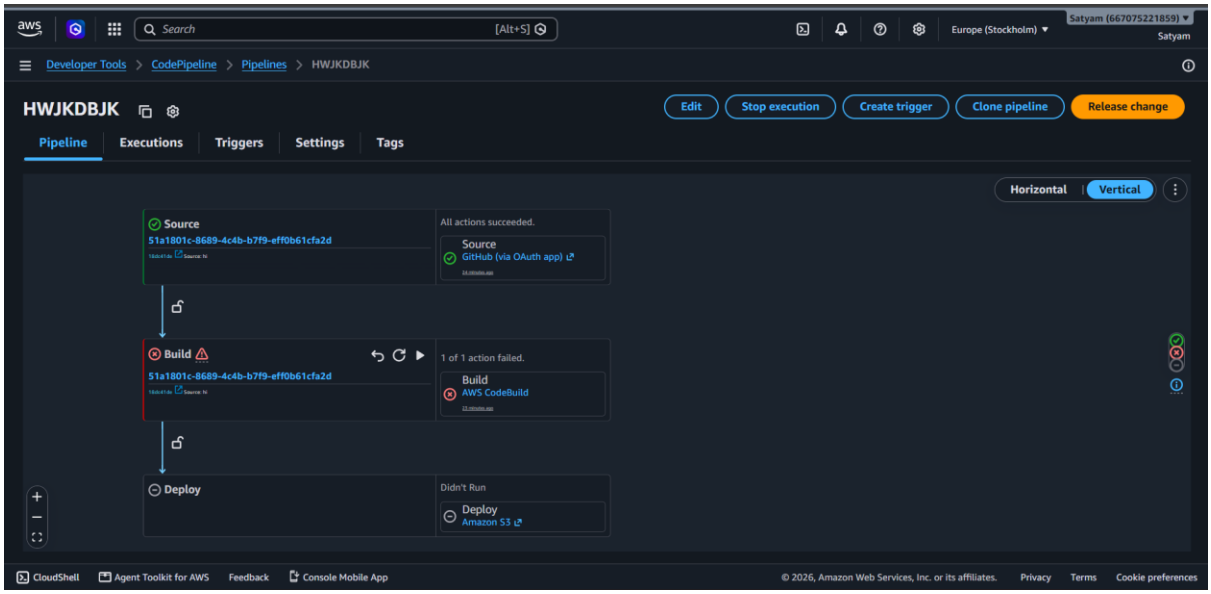
↓

Amazon S3

Click:

Create pipeline

AWS will create and execute the pipeline.




---

**22. Handle Build Permission Issues**

**Step 21: Verify Build Execution**

If the build fails because of insufficient permissions, open:

AWS CodeBuild

→ Build project

→ Build details

→ Service role

The service role opens in AWS IAM.

Open:

Permissions

and select:

Add permissions

→ Attach policies

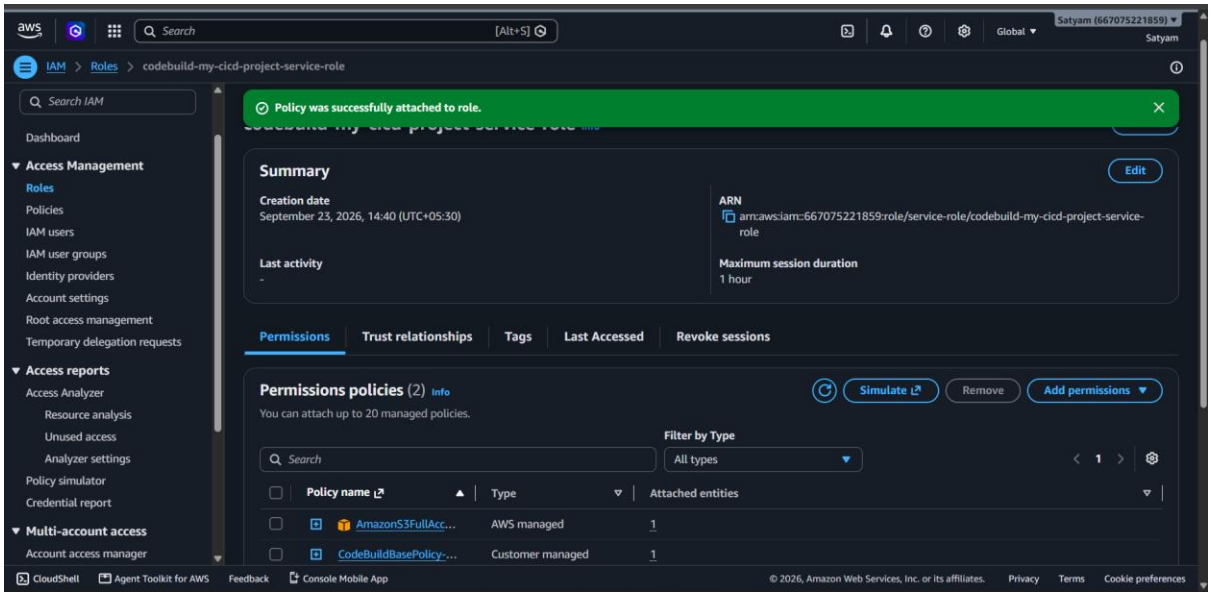
Search for the required S3 permission.

The reference procedure uses:

AmazonS3FullAccess

and attaches it to the CodeBuild service role.

**Note:** For a production system, permissions should normally be restricted to only the required S3 actions/resources rather than granting full S3 access.




---

**23. Retry Pipeline**

**Step 22: Retry Pipeline**

Return to:

AWS CodePipeline

→ my-cicd-pipeline

Retry the failed execution.

Wait for the pipeline to complete.

The expected result is:

Source ✓

Build ✓

Deploy ✓

The pipeline should now execute successfully.

---

**24. Configure Amazon S3 Static Website Hosting**

**Step 23: Open S3 Deployment Bucket**

Search AWS for:

S3

Open the deployment bucket.

Go to:

Properties

Scroll to:

Static website hosting

Click:

Edit

Enable:

Static website hosting

Select:

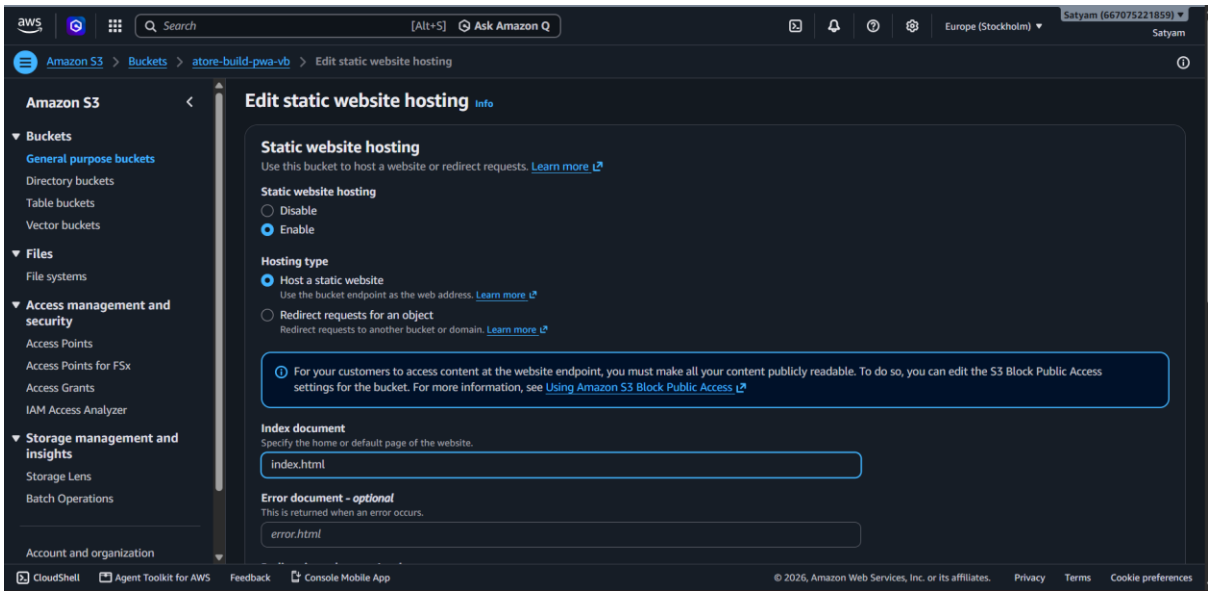
Host a static website

Set the index document to:

index.html

Save the changes.

The reference document follows the same process of enabling static website hosting and specifying index.html.




---

**25. Configure S3 Bucket Policy**

**Step 24: Edit Bucket Policy**

Go to:

S3

→ Deployment Bucket

→ Permissions

→ Bucket policy

Add a policy that permits public GetObject access if your assignment specifically requires a publicly accessible S3 website.

Use your actual bucket name in the resource ARN.

Example:

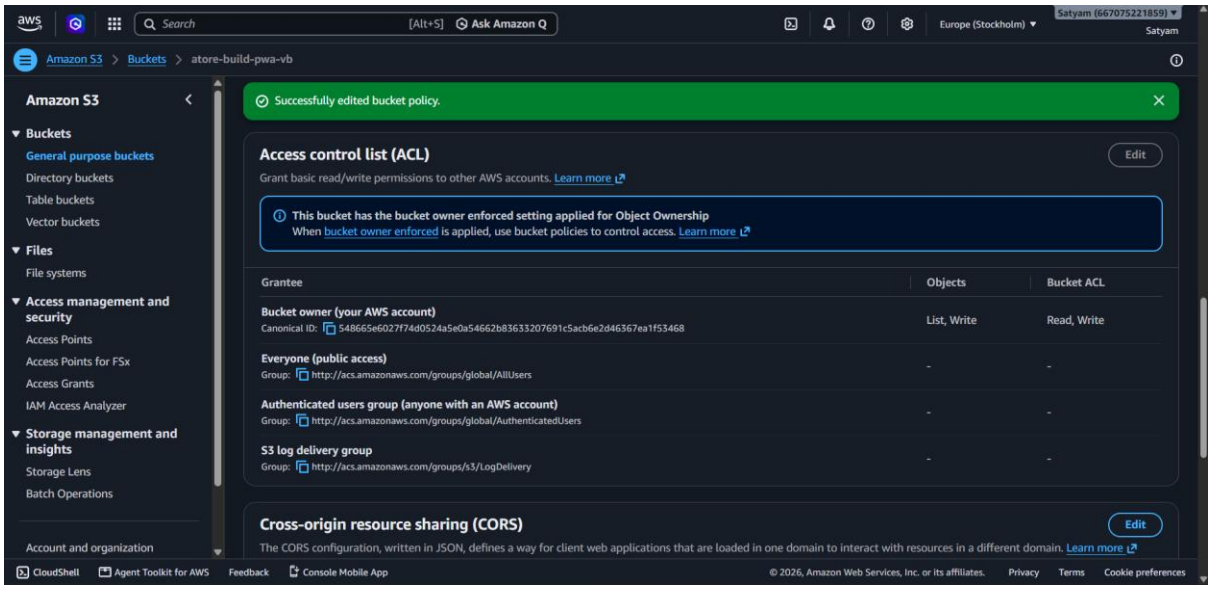
```
{
 "Version": "2012-10-17",
 "Statement": [
  {
   "Sid": "PublicReadGetObject",
   "Effect": "Allow",
   "Principal": "*",
   "Action": "s3:GetObject",
   "Resource": "arn:aws:s3:::YOUR-BUCKET-NAME/*"
  }
 ]
}
```

Replace:

YOUR-BUCKET-NAME

with the actual S3 bucket name.

The reference document uses the same public s3:GetObject pattern.



**26. Open the Deployed Website**

**Step 25: Access Website**

Return to:

S3

→ Deployment Bucket

→ Properties

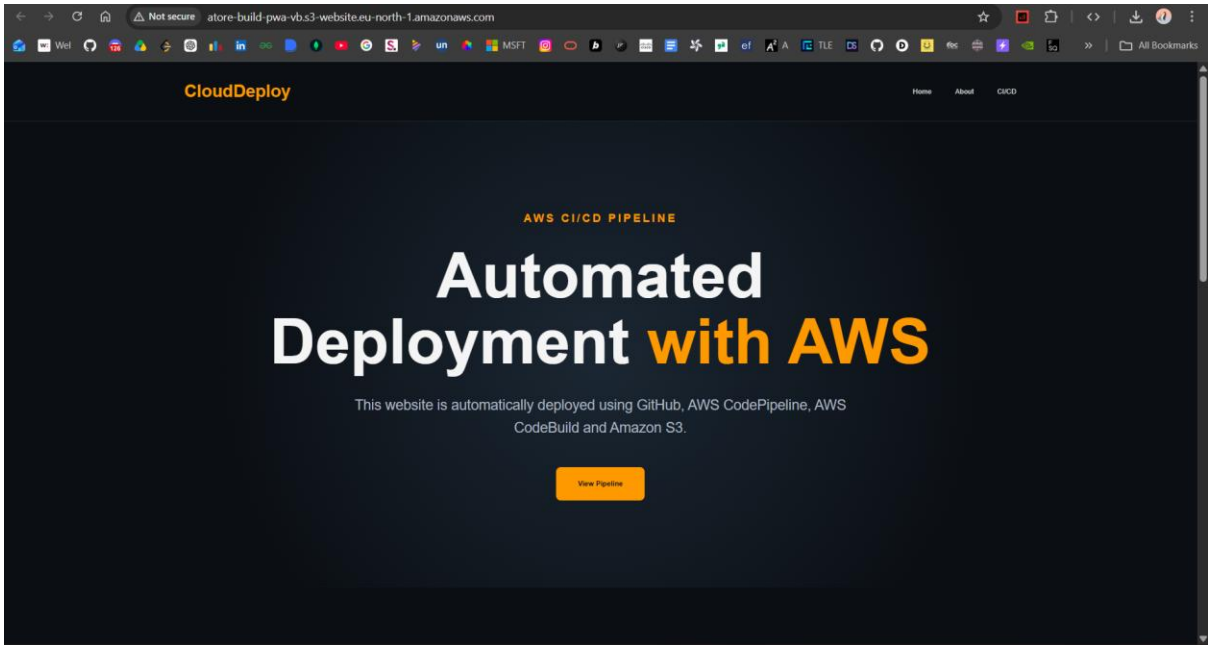
Scroll to:

Static website hosting

AWS displays the **Bucket website endpoint**.

Click the website endpoint.

The deployed static website should open in the browser.




---

**27. Test Automatic Deployment**

**Step 26: Modify the Source Code**

To verify that the CI/CD pipeline works automatically, make a small change to the website.

For example, modify the heading in index.html.

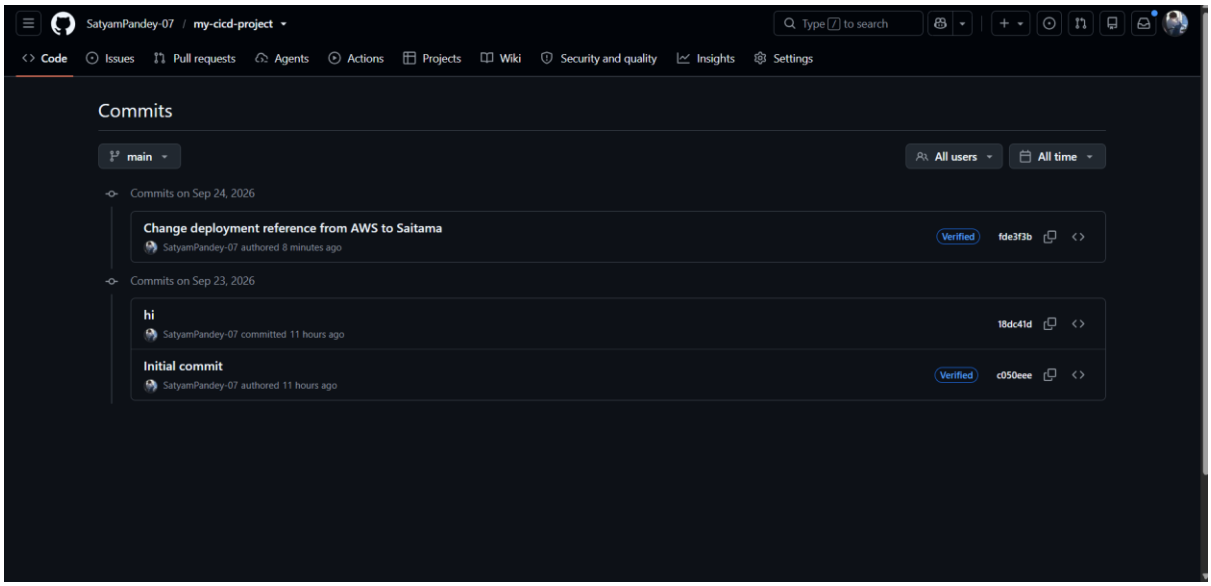
Before:

**<h1>Automated Deployment with AWS</h1>**

After:

**<h1>Automated Deployment with Saitama</h1>**

Commit and push the changes to GitHub.




---

**28. Verify Automatic Pipeline Trigger**

**Step 27: Check CodePipeline**

Open:

AWS CodePipeline

→ my-cicd-pipeline

A new pipeline execution should start after the GitHub change is detected.

The stages should execute in sequence:

GitHub

↓

Source

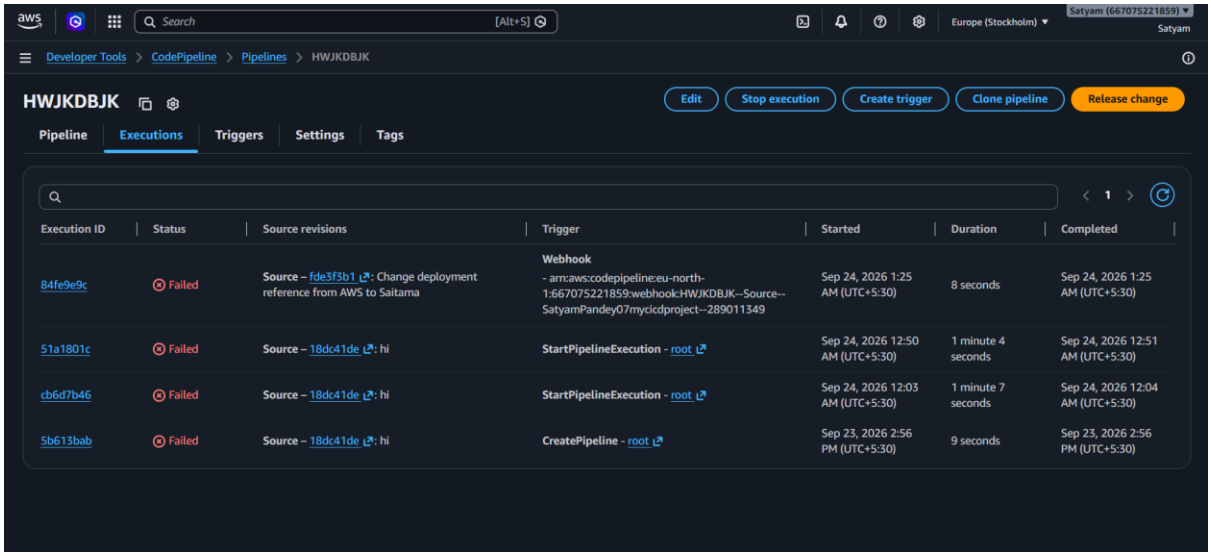
↓

Build

↓

Deploy

Wait until all stages become successful.




---

**29. Verify Updated Website**

**Step 28: Refresh the Website**

Open the S3 static website endpoint again.

Refresh the page.

The changes made in GitHub should now be visible on the deployed website.

This verifies that the complete CI/CD workflow is functioning:

GitHub Change

↓

CodePipeline Trigger

↓

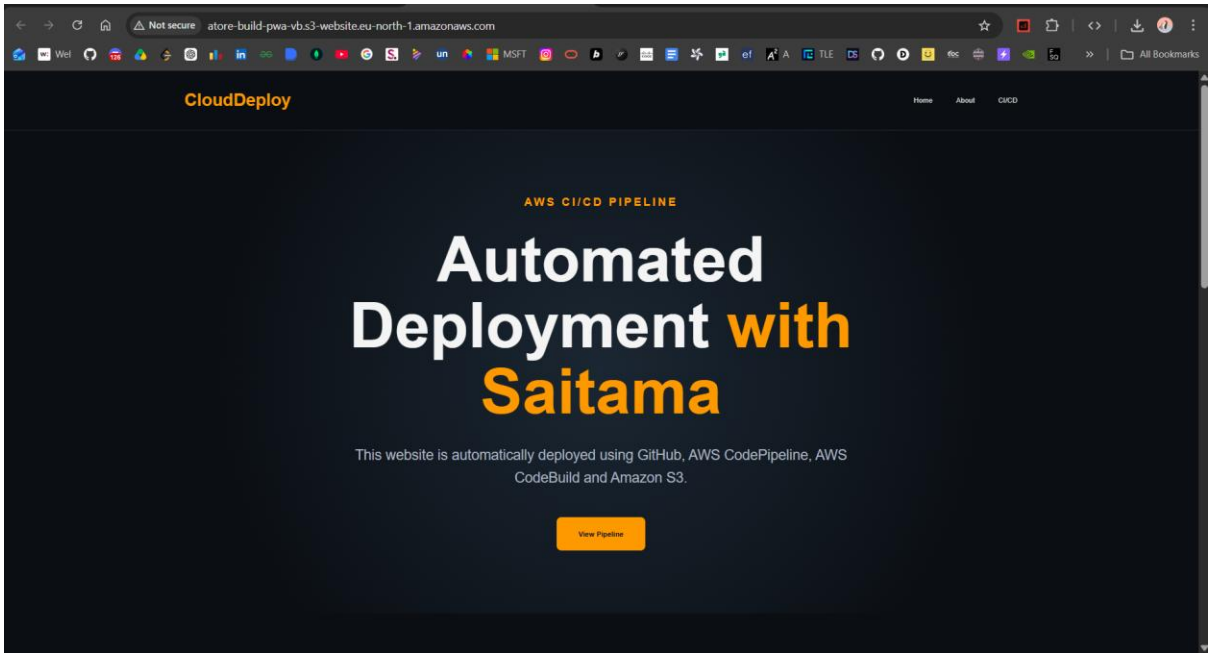
CodeBuild

↓

S3 Deployment

↓

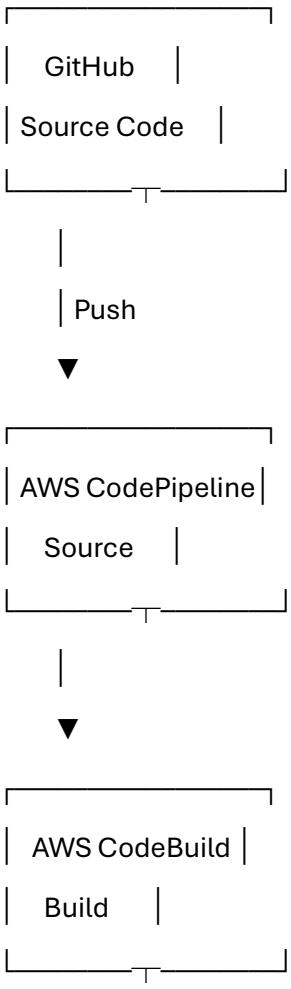
Updated Website

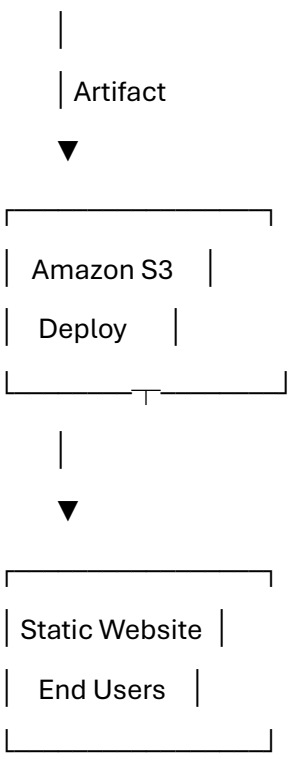



---

### 30. Final CI/CD Workflow

The final implementation can be represented as:






---

**31. Result**

The AWS CI/CD pipeline was successfully configured using **GitHub, AWS CodePipeline, AWS CodeBuild and Amazon S3**.

The source code was maintained in GitHub, automatically processed through AWS CodePipeline and CodeBuild, and deployed to an Amazon S3 static website hosting environment.

A subsequent change made to the GitHub repository triggered a new pipeline execution and updated the deployed website automatically.

---

**32. Conclusion**

Thus, the **AWS CI/CD Pipeline for automated static website deployment** was successfully implemented.

Amazon S3

↓

Static Website

The implementation demonstrates how source-code changes can be automatically built and deployed without manually uploading updated website files to the hosting environment.

---